Lab 2: Investigating Network Traffic over HTTP vs. HTTPS

Objective:

To evaluate and contrast how sensitive data specifically login credentials is transmitted across network protocols, comparing unencrypted HTTP traffic against encrypted HTTPS traffic using Wireshark.

Analyzing Unencrypted HTTP Traffic :

1. Initiating the Login Sequence:

The assessment starts by navigating to the "VulnLab Bank Login" page via a standard HTTP connection. Because the connection lacks encryption, the web browser displays a "Not Secure" warning. After submitting the username admin and a test password, the user successfully authenticates and is redirected to the dashboard, which exposes sensitive financial records over the insecure channel.

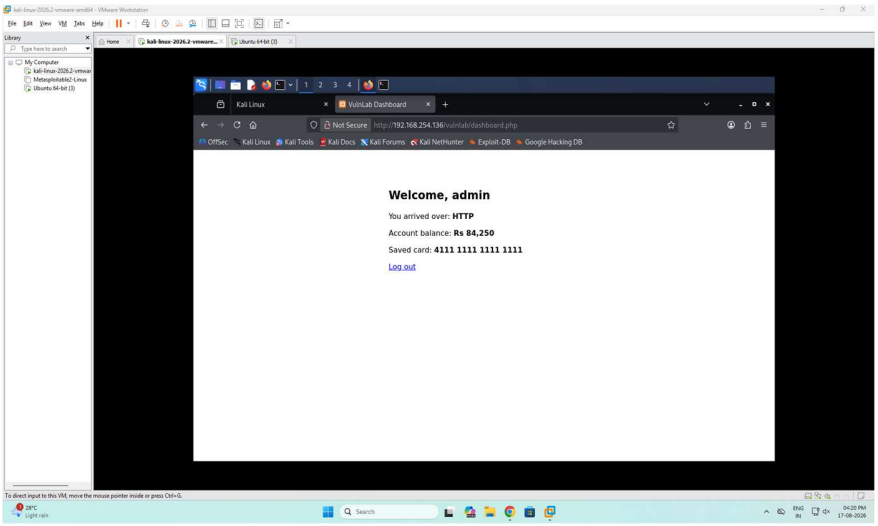

Fig. 1: Insecure HTTP connection displaying the VulnLab dashboard

2. Capturing Network Packets:

By deploying Wireshark to monitor network activity, we apply an HTTP POST filter to isolate the exact packet generated when the user submits their login form to the server.

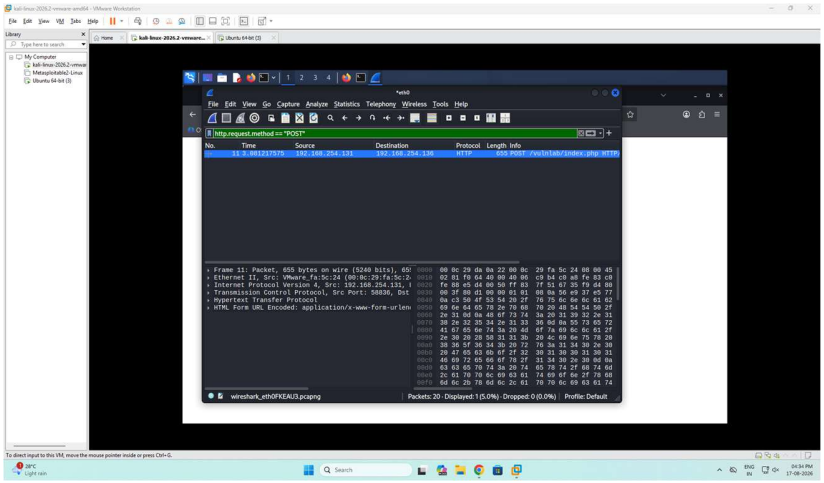

Fig. 2: Isolating the HTTP POST request in Wireshark

3. Identifying the Exposure:

Reviewing the TCP stream of this captured POST request highlights the fundamental weakness of HTTP:

Cleartext Data: All payload data travels across the network without any encryption.

Compromised Credentials: The exact string username=admin and password=SuperSecret123 is fully readable to anyone intercepting the traffic. Consequently, an attacker executing a Man-in-the-Middle (MitM) attack could effortlessly steal these login details.

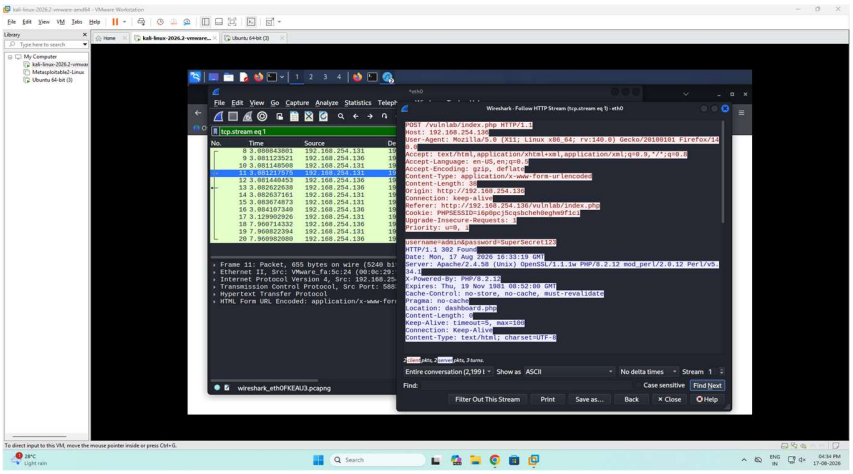

Fig. 3: TCP stream revealing plaintext HTTP credentials

Analyzing Encrypted HTTPS Traffic:

1. Initiating the Login Sequence:

Next, the user attempts to reach the identical portal using HTTPS. Since the test environment utilizes a self-signed certificate instead of one verified by a recognized Certificate Authority, the browser generates a "Potential Security Risk" alert. Once the user bypasses the warning and accepts the certificate exception, they successfully authenticate and access the secure version of the dashboard.

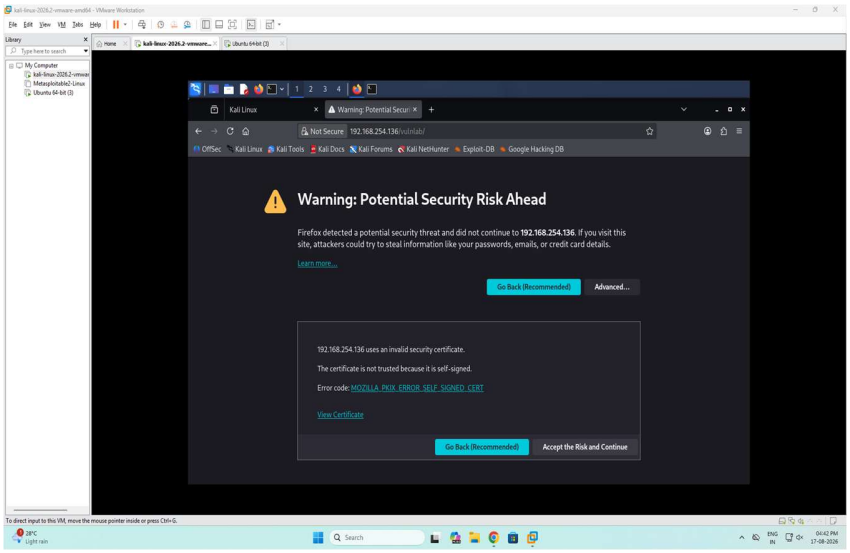

Fig. 4: Browser warning triggered by a self-signed HTTPS certificate

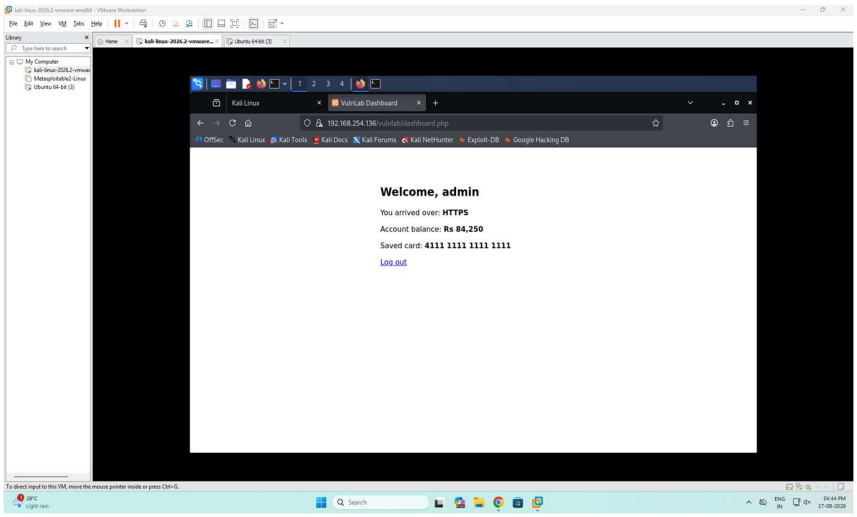

Fig. 5: Successful dashboard access over a secure HTTPS connection

2. Packet Inspection Under Encryption:

When conducting the same Wireshark analysis on this secure session, filtering for http.request.method == "POST" produces zero captured packets. This occurs because all standard HTTP communications are now wrapped inside a secure tunnel.

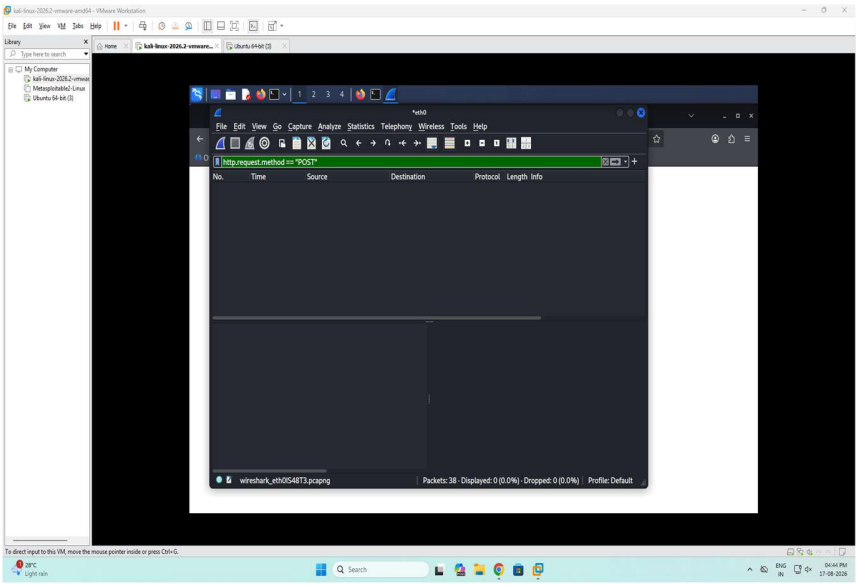

Fig. 6: Wireshark returning no HTTP POST results during the HTTPS session

3. Evaluating TLS Encryption:

Instead of exposing raw HTTP data, Wireshark now displays Transport Layer Security (TLS) traffic. The capture logs the initial handshake process—including the Client Hello, Server Hello, and Change Cipher Spec—used by the client and server to negotiate encryption parameters.

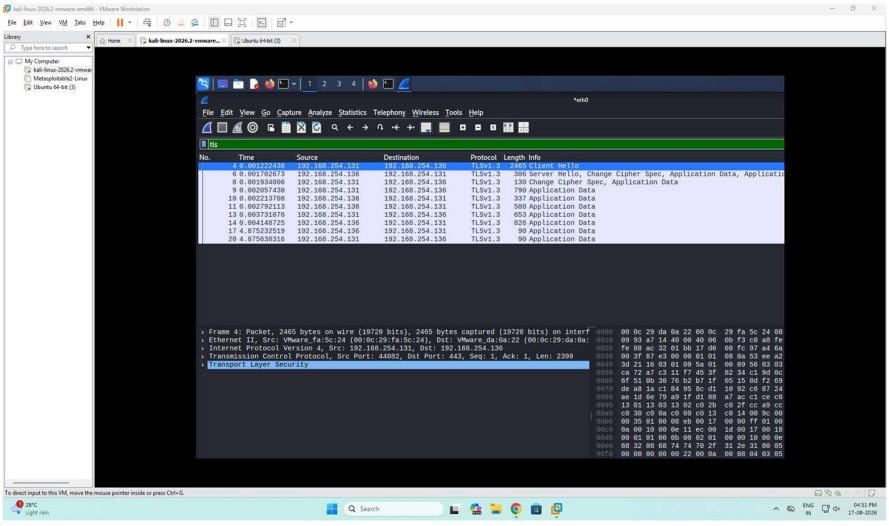

Fig. 7: Secure communication establishment via the TLS handshake

- Obscured Payload: Looking closely at the data packets, the underlying content is entirely illegible. Wireshark accurately categorizes this data as "Encrypted Application Data."

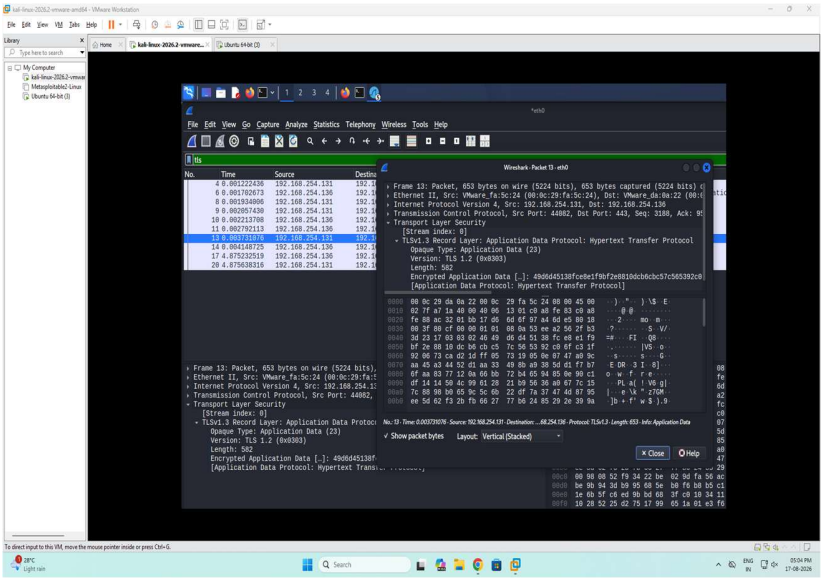

Fig. 8: Wireshark displaying unreadable, encrypted application data

- Verified Security: To confirm that the credentials cannot be extracted, a string search in Wireshark for the payload (frame contains "SuperSecret123") returns absolutely no matches. The encryption mechanism effectively shields the sensitive information from eavesdroppers.

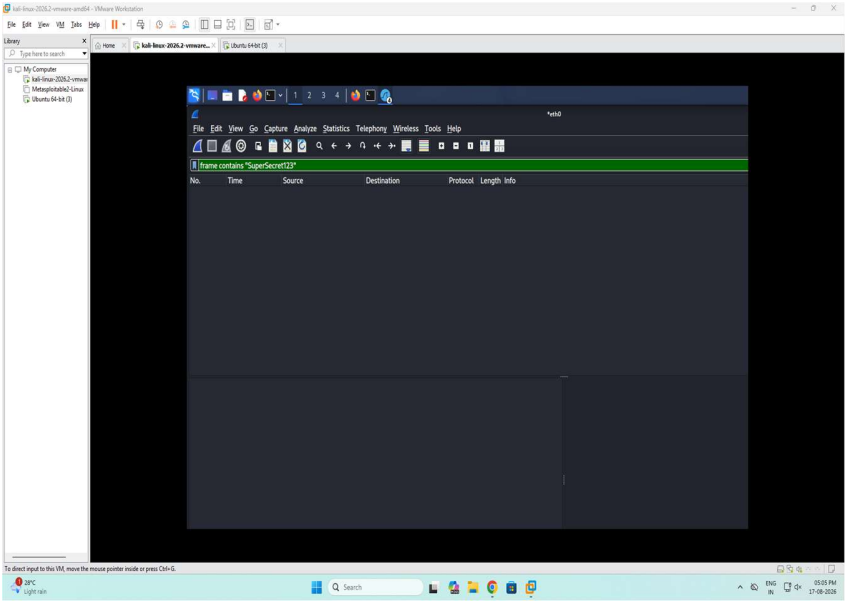

Fig. 9: Search query confirming the password is safely hidden from packet inspection